```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import pandas as pd
import numpy as nm
```

```
iris  = pd.read_csv("/content/drive/MyDrive/Machine Learning/IRIS.csv")
iris
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

```
iris.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
iris.tail()
```

	sepal_length	sepal_width	petal_length	petal_width	species
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

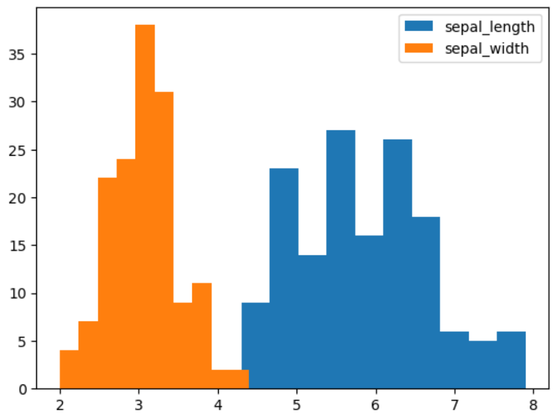
```
iris.count()
```

```
sepal_length    150
sepal_width     150
petal_length    150
petal_width     150
species         150
dtype: int64
```

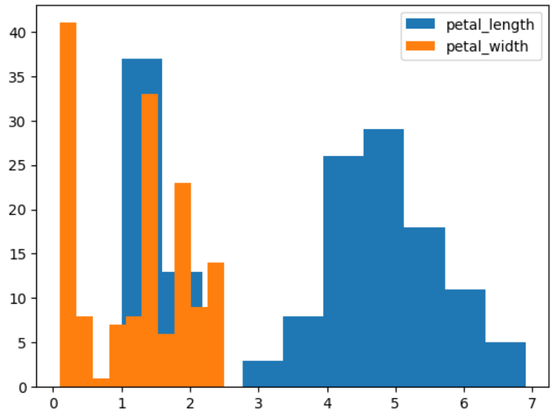
```
categorical_cols = iris.select_dtypes(include=['object']).columns.tolist()
categorical_cols
```

```
['species']
```

```
import matplotlib.pyplot as plt

plt.hist(iris['sepal_length'], label='sepal_length')
plt.hist(iris['sepal_width'], label='sepal_width')
plt.legend()
plt.show()
```




```
#Histogram
plt.hist(iris['petal_length'], label='petal_length')
plt.hist(iris['petal_width'], label='petal_width')
plt.legend()
plt.show()
```




```
#Label Encoding
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in categorical_cols:
  iris[col] = le.fit_transform(iris[col])

iris.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
#Predictor and target variable

X = iris.iloc[:, :4]
y = iris.iloc[:, 4]
X
y
```

```
0      0
1      0
2      0
3      0
4      0
      ..
145    2
146    2
147    2
148    2
149    2
Name: species, Length: 150, dtype: int64
```

```
# x=iris.drop(column['species'])
# y=iris['species']

#Split data

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_train
```

	sepal_length	sepal_width	petal_length	petal_width
81	5.5	2.4	3.7	1.0
133	6.3	2.8	5.1	1.5
137	6.4	3.1	5.5	1.8
75	6.6	3.0	4.4	1.4
109	7.2	3.6	6.1	2.5
...	...	...	...	...
71	6.1	2.8	4.0	1.3
106	4.9	2.5	4.5	1.7
14	5.8	4.0	1.2	0.2
92	5.8	2.6	4.0	1.2
102	7.1	3.0	5.9	2.1

105 rows × 4 columns

```
X_test
```

	sepal_length	sepal_width	petal_length	petal_width
73	6.1	2.8	4.7	1.2
18	5.7	3.8	1.7	0.3
118	7.7	2.6	6.9	2.3
78	6.0	2.9	4.5	1.5
76	6.8	2.8	4.8	1.4
31	5.4	3.4	1.5	0.4
64	5.6	2.9	3.6	1.3
141	6.9	3.1	5.1	2.3
68	6.2	2.2	4.5	1.5
82	5.8	2.7	3.9	1.2
110	6.5	3.2	5.1	2.0
12	4.8	3.0	1.4	0.1
36	5.5	3.5	1.3	0.2
9	4.9	3.1	1.5	0.1
19	5.1	3.8	1.5	0.3
56	6.3	3.3	4.7	1.6
104	6.5	3.0	5.8	2.2
69	5.6	2.5	3.9	1.1
55	5.7	2.8	4.5	1.3
132	6.4	2.8	5.6	2.2
29	4.7	3.2	1.6	0.2
127	6.1	3.0	4.9	1.8
26	5.0	3.4	1.6	0.4
128	6.4	2.8	5.6	2.1
131	7.9	3.8	6.4	2.0
145	6.7	3.0	5.2	2.3
108	6.7	2.5	5.8	1.8
143	6.8	3.2	5.9	2.3
45	4.8	3.0	1.4	0.3
30	4.8	3.1	1.6	0.2
22	4.6	3.6	1.0	0.2

```
y_train
```

```
81     1
133    2
137    2
75     1
109    2
```

```
        ..
71     1
106    2
14     0
92     1
102    2
Name: species, Length: 105, dtype: int64
```

```
y_test
```

```
73     1
18     0
118    2
78     1
76     1
31     0
64     1
141    2
68     1
82     1
110    2
12     0
36     0
9      0
19     0
56     1
104    2
69     1
55     1
132    2
29     0
127    2
26     0
128    2
131    2
145    2
108    2
143    2
45     0
30     0
22     0
15     0
65     1
11     0
42     0
146    2
51     1
27     0
4      0
32     0
142    2
85     1
86     1
16     0
10     0
Name: species, dtype: int64
```

```
# built model using Logistic regression

from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_pred
```

```
array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1, 2, 0, 0, 0, 0, 1, 2, 1, 1, 2, 0, 2,
       0, 2, 2, 2, 2, 2, 0, 0, 0, 0, 1, 0, 0, 2, 1, 0, 0, 0, 2, 1, 1, 0,
       0])
```

```
# train model using training data

model.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

```
#Evaluate the model's performance on test data using 'accuracy' measure

from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(accuracy)
```

```
1.0
```

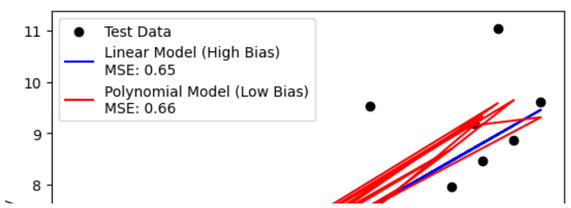
```
#Evaluate the model's performance on traning data using 'accuracy' measure

y_pred_train = model.predict(X_train)
accuracy_train = accuracy_score(y_train, y_pred_train)
print(accuracy_train)
```

```
0.9619047619047619
```

```
# find your model is overfitting  , underfitting or well train model

# To determine whether your model is overfitting, underfitting, or well-trained, you can compare the training and testing accuracy scores.

# - **Overfitting:** If the training accuracy is significantly higher than the testing accuracy, it indicates that your model has learned the training data too well and is not generalizing well to
# - **Underfitting:** If both the training and testing accuracy scores are low, it suggests that your model is not learning the data effectively. This is known as underfitting.
# - **Well-trained:** If the training and testing accuracy scores are relatively close and reasonably high, it indicates that your model has learned the data well and is generalizing well to unseen

# In your case, the training accuracy is 1.0, while the testing accuracy is 0.9778. This suggests that your model might be slightly overfitting the training data. However, the difference is not sig

# To further investigate potential overfitting, you can try the following:

# * **Increase the size of your training data:** This can help your model learn more general patterns and reduce overfitting.
# * **Reduce the complexity of your model:** For example, you can try using a simpler model architecture or reducing the number of features used.
# * **Use regularization techniques:** Regularization techniques can help prevent overfitting by penalizing the model for having large weights.

# By experimenting with these techniques, you can fine-tune your model and achieve better generalization performance.
```

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score


X, y = make_classification(n_samples=1000, n_features=20, random_state=42)


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
train_predictions = model.predict(X_train)
train_accuracy = accuracy_score(y_train, train_predictions)


test_predictions = model.predict(X_test)
test_accuracy = accuracy_score(y_test, test_predictions)

print(f"Training Accuracy: {train_accuracy:.4f}")
print(f"Validation Accuracy: {test_accuracy:.4f}")


if train_accuracy < test_accuracy:
    print("The model may be overfitting.")
elif train_accuracy < test_accuracy:
    print("The model may be underfitting.")
else:
    print("The model is well-trained.")
```

```
Training Accuracy: 1.0000
Validation Accuracy: 0.9000
The model is well-trained.
```

```
#model have high bias or low bias

# The model has low bias

#  A model with high bias tends to underfit the data, meaning it makes many incorrect predictions on both the training and test sets.
#  A model with low bias tends to overfit the data, meaning it makes very few incorrect predictions on the training set but more incorrect predi
# In this case, the model has a high training accuracy (1.0) and a slightly lower testing accuracy (0.9778). This suggests that the model is ove
```

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

# Generate synthetic data with a quadratic relationship
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Model with High Bias (Linear Regression)
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Model with Low Bias (Polynomial Regression)
poly_features = PolynomialFeatures(degree=10)
X_train_poly = poly_features.fit_transform(X_train)
poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)

# Evaluate the models on the test set
X_test_poly = poly_features.transform(X_test)
y_pred_linear = linear_model.predict(X_test)
y_pred_poly = poly_model.predict(X_test_poly)

# Calculate Mean Squared Error (MSE)
mse_linear = mean_squared_error(y_test, y_pred_linear)
mse_poly = mean_squared_error(y_test, y_pred_poly)

# Plot the models and data
plt.scatter(X_test, y_test, color='black', label='Test Data')
plt.plot(X_test, y_pred_linear, color='blue', label=f'Linear Model (High Bias)\nMSE: {mse_linear:.2f}')
plt.plot(X_test, y_pred_poly, color='red', label=f'Polynomial Model (Low Bias)\nMSE: {mse_poly:.2f}')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.show()
```